ЛР3. Построение локальной системы RAG для технической документации

- 1/ Цель работы
- 2/ Теория
  - 2.1/ Векторное представление слов (word embedding, эмбеддинги)
  - Индексация
  - "Нарезка" (chunking)
  - 2.2/ Архитектура RAG: «Поиск → Контекст → Ответ»
- 3/ Ход работы
  - Подготовка среды
    - Установка Ollama
  - Шаг 1 - загрузка и нарезка текста
    - Нарезка документа (Splitting documents)
  - Шаг 2 - эмбеддинги и векторное хранилище
    - Семантический поиск
  - Шаг 3 - создание RAG-агента
  - ЗАДАНИЕ
- Дополнительная информация
- Форма отчета

1/ Цель работы

Освоить цикл создания RAG-системы (*RAG-agent*) с помощью LangChain и Ollama: от обработки текста до генерации ответа на основе найденного контекста. Для примера будем использовать небольшой справочник по основным командам Git (система контроля версия).

Инструменты: Python, LangChain, Ollama

2/ Теория

Чтобы построить работающую систему, нужно разобраться, как текст превращается в "смысловые координаты" и как модель "читает" предоставленные ей документы.

2.1/ Векторное представление слов (word embedding, эмбеддинги)

Текст как точка в пространстве

Вместо того чтобы искать точные совпадения слов (как в обычном поиске Ctrl+F), мы представляем фрагменты текста в виде векторов — наборов чисел.

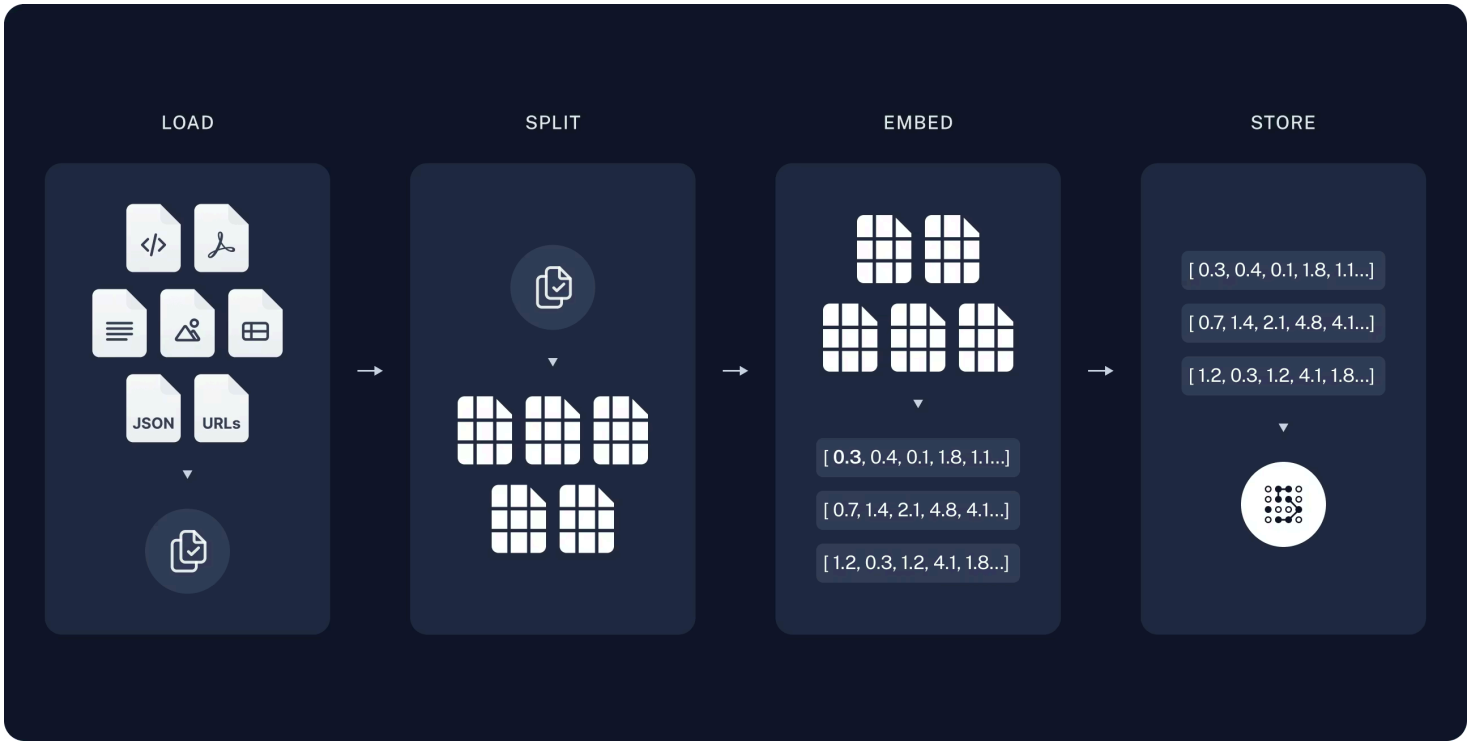
- **Суть**: В многомерном пространстве векторы текстов со схожим смыслом (например, "как создать ветку Git" и "команда для новой ветки Git") будут находиться близко друг к другу.
- **Метрика**: Близость обычно измеряется через **косинусное сходство**. Чем меньше угол между векторами, тем выше семантическое сходство.

Индексация

Обычно индексация работает следующим образом:

- Этап 1 - загрузка данных (load): текстовые документы, сайты, PDF-файлы. Загрузка делается с помощью специальных модулей **Document Loaders**
- Этап 2 - нарезка (split, chanking): текстовые разделители разбивают большие документы на более мелкие фрагменты - **chunks**. Это полезно как для индексации данных, так и для их передачи в модель, поскольку поиск в больших фрагментах затруднен и они не помещаются в ограниченное контекстное окно модели.
- Этап 3 - хранение (store): нам нужно где-то хранить и индексировать наши разбиения, чтобы их можно было искать позже. Это часто делается с помощью векторных базы данных (VectorStore, vector database) и модели встраивания (**Embeddings model**).

> **Info**
>
> **Модели встраивания** (**Embeddings model**) преобразуют исходный текст, такой как предложение, абзац или заметку, в вектор чисел фиксированной длины, который отражает его семантическое значение. Эти векторы позволяют компьютерам сравнивать и осуществлять поиск текста на основе значения, а не точных слов.



"Нарезка" (chunking)

Мы не можем просто загрузить в базу весь справочник целиком. У каждой модели есть **контекстное окно** (предел текста, который она «видит» за раз), а векторный поиск работает эффективнее с небольшими, но законченными по смыслу фрагментами — **чанками** (chunks).

Вот три главных правила качественного чанкинга:

- **Размер фрагмента**. Если чанк слишком маленький (например, одно слово), он теряет смысл. Если слишком большой — в векторе «размывается» конкретика. Для технических

справочников обычно выбирают 500–1000 символов.

- **Наложение (Overlap)**. Мы делаем так, чтобы конец первого чанка немного повторял начало второго (обычно на 10–20%). Это нужно, чтобы важная мысль не оказалась разрезанной пополам на стыке.
- **Семантические границы**. Профессионалы стараются резать текст там, где это логично: на концах абзацев, заголовков или предложений, а не в середине слова.

В данной работе предлагается использовать `RecursiveCharacterTextSplitter` — один из самых популярных инструментов в LangChain для подготовки данных.

В отличие от простых разделителей, которые режут текст ровно через каждые N символов (и могут разорвать слово или команду Git на середине), этот класс действует «умнее». Он использует иерархию разделителей, чтобы сохранить смысловую целостность фрагмента.

Как работает алгоритм
По умолчанию он пробует разделять текст в следующем порядке:

1. **Двойной перенос строки** (`\n\n`) — чтобы сохранить абзацы целиком.
2. **Одиночный перенос строки** (`\n`) — чтобы не разрывать строки внутри абзаца.
3. **Пробел** ( ) — чтобы не разрывать слова.
4. **Пустая строка** (`""`) — если фрагмент всё равно слишком велик, он режет по символам.

Почему это важно для нашего справочника Git?
Если мы будем нарезать справочник Git простым методом, мы рискуем получить один чанк с описанием команды `git commit` , а параметры этой команды — в другом чанке. `RecursiveCharacterTextSplitter` постарается удержать команду и её описание в одном блоке, так как они, скорее всего, разделены только пробелами или одной новой строкой.

Основные параметры, которые мы будем настраивать:

- `chunk_size` : Максимальный размер чанка (например, 300 символов).
- `chunk_overlap` : «Нахлест» между чанками (например, 50 символов), чтобы контекст не терялся на границах.

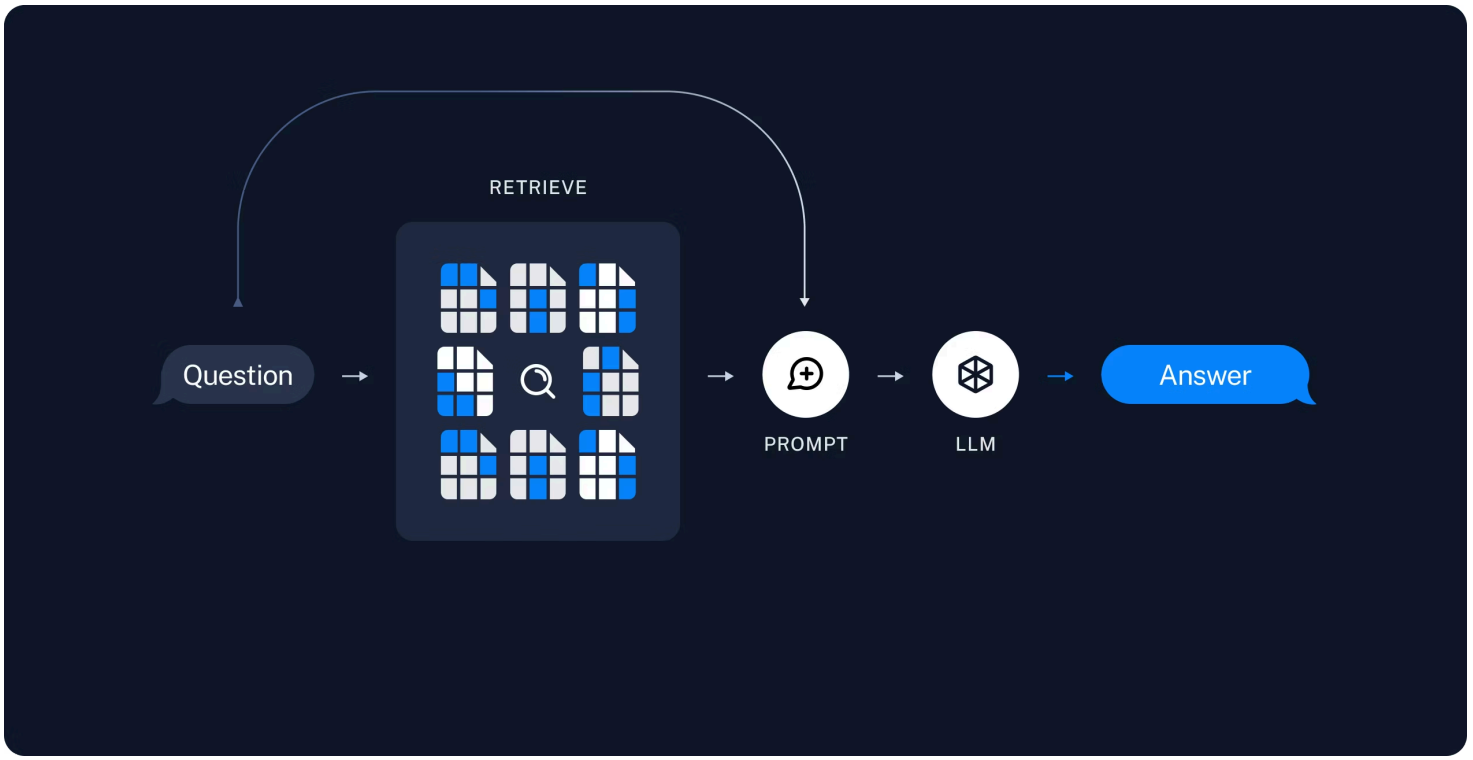
2.2/ Архитектура RAG: «Поиск → Контекст → Ответ»

Приложения **RAG (Retrieval and generation**) обычно работают следующим образом:

1. **Retrieve** (извлечение): при вводе данных пользователем соответствующие разделы извлекаются из векторного хранилища с помощью средства поиска.
2. **Generate** (генерирование): LLM выдает ответ, используя prompt, который включает в себя как вопрос, так и полученные данные

> **Info**
>
> Можно встретить другую расшифровку **RAG - Retrieval-Augmented Generation**. Здесь отдельно выделяется процесс **Augmentation (обогащение)** - найденные фрагменты

добавляются в промпт (инструкцию) для модели, а модель уже пишет ответ опираясь только на предоставленный текст



3/ Ход работы

Подготовка среды

Для выполнения данной лабораторной работы установите следующие зависимости:

```
pip install -U langchain langchain_community langchain_text_splitters
```

Установка Ollama

В данной работе используется запуск LLM на локальной инфраструктуре через платформу **Ollama**

- сайт: https://ollama.com/
- документация: https://docs.ollama.com/
- список моделей: https://ollama.com/search
- клиентская библиотека для Python: https://github.com/ollama/ollama-python

Ollama обеспечивает развертывание и администрирование локальных LLM. В качестве локальных моделей используются легкие LLM с небольшим числом параметров. Для запуска более мощных моделей Ollama также предоставляет доступ к облачным моделям

Запуск Ollama
Для Windows вместе в комплекте установки Ollama идет графический интерфейс, через который можно загружать и запускать локальные LLM.

Работа через CLI
CLI интерфейс Ollama позволяет также зазгружать, запускать и администрировать LLM.

Рассмотрим основные команды (https://docs.ollama.com/cli):

- Вывод списка загруженных моделей

```
ollama list
```

- Загрузка модели

```
ollama pull qwen3:1.7b
```

В данном примере мы вводим команду загрузки open source модели Qwen с *1.7 млрд. параметров*

> **Info**
>
> Если ваш ПК потянет больше, можете загрузить более увесистые. Главное убедитесь, что модели поддерживают tools https://ollama.com/search?c=tools

- Удаление модели

```
ollama rm qwen3:1.7b
```

- Запуск модели в терминале

```
ollama run qwen3:1.7b
```

> **Примечание**
>
> В командах выше мы установили модель которая будет "мозгом" нашего агента, именно она будет формировать ответ на основе контекста, предоставленного RAG. Но нам также понадобиться модель встраивания (embedding model). Будем использовать модель от Google `embeddinggemma` (https://ollama.com/library/embeddinggemma). **Установите ее.**

Шаг 1 - загрузка и нарезка текста

Текст документации представлен в Markdown-файле `git-doc.md`. Загрузите его:

```
with open('git-doc.md', encoding='utf-8') as f:
    git_guide = f.read()

print(f"Total characters: {len(git_guide)}")
```

```
Total characters: 11550
```

Прочитаем первый отрывок из документации:

```
print(git_guide[:500])
```

```
## Приветствие

Итак, вы решили использовать **git**, но не знаете с чего начать? Эту документацию можно рассматривать в качестве настольного пособия и обращаться к ней по многим возникающим вопросам. Для начала вам следует создать свой [`проект`](https://gitflic.ru/project/create). Когда вы создали свой новый репозиторий, необходимо подготовить локальное рабочее пространство. Используйте консоль на вашем ПК для работы.

> Для большего удобства пользователям Windows мы рекомендуем установить отд
```

Нарезка документа (Splitting documents)

Наш загруженный документ содержит более 11 тыс. символов, что слишком много для того, чтобы поместиться в контекстном окне многих моделей. Даже для тех моделей, которые могли бы разместить всю публикацию в своем контекстном окне, модели могут испытывать трудности с поиском информации в очень длинных вводных данных.

Для работы нам понадобится библиотека `langchain-text-splitters`. Вот как выглядит скрипт, который берет сырой текст и готовит его к превращению в векторы:

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Инициализируем разделитель
# Мы ставим небольшой chunk_size, чтобы каждый чанк содержал 1-2 команды
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=300,
    chunk_overlap=50
)

# Разрезаем текст на документы
chunks = text_splitter.split_text(git_guide)

print(f"Split blog post into {len(chunks)} sub-documents.")
```

```
Split blog post into 60 sub-documents.
```

Шаг 2 - эмбеддинги и векторное хранилище

Теперь, когда у нас есть кусочки текста, компьютер должен их «понять». Мы не можем просто искать слова — нам нужно превратить каждый чанк в **эмбеддинг** (вектор из чисел). Нам нужно проиндексировать наши 60 фрагментов текста, чтобы мы могли выполнять поиск по ним во время выполнения.

Для векторизации нам понадобится модуль `OllamaEmbeddings` и модель встраивания выбранная раньше.

```python
from langchain_ollama import OllamaEmbeddings

# Инициализируем модель для создания эмбеддингов через Ollama
# Убедитесь, что модель эмбеддинга скачана: ollama pull embeddinggemma
embeddings = OllamaEmbeddings(model="embeddinggemma")
```

В качестве векторного хранилища используем `InMemoryVectorStore` (сохраняется в оперативной памяти).

```python
from langchain_core.vectorstores import InMemoryVectorStore
vector_store = InMemoryVectorStore(embeddings)

ids = vector_store.add_texts(texts=chunks)
```

Прочитаем индексы документов (для примеры первые 3)

```python
print(ids[:3])
```

```
['0d4d05ce-e168-4fd6-ba1c-c4822295a653', 'ef307d23-6902-47c0-a62e-9c7795babc44',
'9933fe7b-51f9-4407-b8be-cf957ac654fa']
```

Мы можем извлечь содержимое chank по индексу

```python
print(vector_store.get_by_ids(ids)[1].page_content)
```

```
Итак, вы решили использовать **git**, но не знаете с чего начать? Эту документацию
можно рассматривать в качестве настольного пособия и обращаться к ней по многим
возникающим вопросам. Для начала вам следует создать свой [`проект`]
(https://gitflic.ru/project/create). Когда вы создали свой новый
```

Семантический поиск

Мы можем выполните семантический поиск по созданной векторной базе данных. Следующий код демонстрирует эту возможность:

```python
results = vector_store.similarity_search(
    "git pull?" # Наш вопрос
)
print(results[0].page_content)
```

Получим в ответ chank, максимально близкий по смыслу

```
Опция `git pull --no-commit` выполняет объединение, но не создает новый коммит со
слитым содержимым:

git pull --no-commit &lt;remote&gt;

### git push
```

Шаг 3 - создание RAG-агента

Одна из вариантов приложения RAG - это простой агент с инструментом, который извлекает информацию. Мы можем создать минимальный агент RAG, у которого будет инструмент `langchain.tools` извлечение данных и нашей векторной базы данных. Инструментом будет является функция, выполняющая семантический поиск по базе данных на основе запроса:

```
from langchain.tools import tool

@tool(response_format="content_and_artifact")
def retrieve_context(query: str):
    """Извлечение информации, которая поможет ответить на запрос."""
    retrieved_docs = vector_store.similarity_search(query, k=2)
    serialized = "\n\n".join(
        (f"Source: {doc.metadata}\nContent: {doc.page_content}")
        for doc in retrieved_docs
    )
    return serialized, retrieved_docs
```

Теперь давайте импортируем LLM, которая будет обрабатывать информацию и выдавать ответ

```
from langchain_ollama.chat_models import ChatOllama

model = ChatOllama(model="qwen3:1.7b") # вписываем название модели, которая установлена локально в Ollama
```

Далее создаем **агента**. Для него мы также пропишем **системный промпт** для настройки поведения LLM.

```
from langchain.agents import create_agent

tools = [retrieve_context] # указываем в инструментах наш RAG

system_prompt = (
    "У тебя есть доступ к инструменты извлечения дополнительной информации."
    "Используй этот инструмент для ответа на пользовательский вопрос"
)

agent = create_agent(model, tools, system_prompt=system_prompt)
```

Введем пользовательский запрос

```
query = (
    "Что делает команда git init?\n\n"
    "Как только получите ответ, приведите несколько примеров использования этой команды"
    "Отвечай на русском языке"
)
```

Запускаем агента и читаем ответ

```
res = agent.invoke({"messages": [{"role": "user", "content": query}]})
res['messages'][-1].pretty_print()
```

ЗАДАНИЕ

Задание 1

Вам необходимо модернизировать RAG-агента путем добавления в него пайплайна загрузки и парсинга информации с web-страницы по указанному URL. В Langchain есть для этого инструменты (см. `WebBaseLoader`) . Для парсинга HTML-страницы можно использовать `BeautifulSoup` .

В конечном итоге вы должны получить RAG-агента, который отвечает на вопросы СТРОГО (пропишите это в системном промпте) по содержимому web-страницы.

Дополнительная информация

- https://docs.langchain.com/
- https://docs.langchain.com/oss/python/langchain/rag
- https://docs.langchain.com/oss/python/langchain/knowledge-base
- https://docs.langchain.com/oss/python/langchain/retrieval

Форма отчета

1. Python script, Jupyter Notebook (или Google Colab) с ходом работы (векторизация, индексация, RAG)
2. Текстовый отчет работы RAG-агента по заданию 1. Продемонстрируйте возможности агента на нескольких примерах